problem_2

March 9, 2026

1 Mouhamed Cisse

[47]:
```
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from keras.models import Sequential
import pandas as pd
```

[48]:
```
import h5py

with h5py.File("Problem 2/train_happy.h5", "r") as f:
    print(list(f.keys()))
    X_train = f['train_set_x'][:]
    Y_train = f['train_set_y'][:]
with h5py.File("Problem 2/test_happy.h5", "r") as f:
    print(list(f.keys()))
    X_test = f['test_set_x'][:]
    Y_test = f['test_set_y'][:]

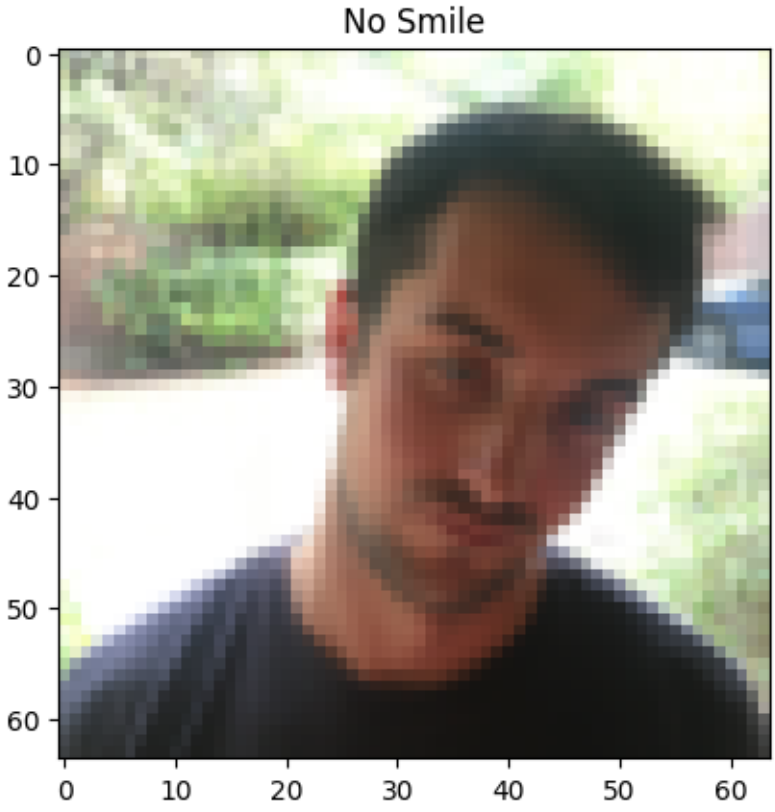
print("Train shape:", X_train.shape, Y_train.shape)
print("Test shape:", X_test.shape, Y_test.shape)
```

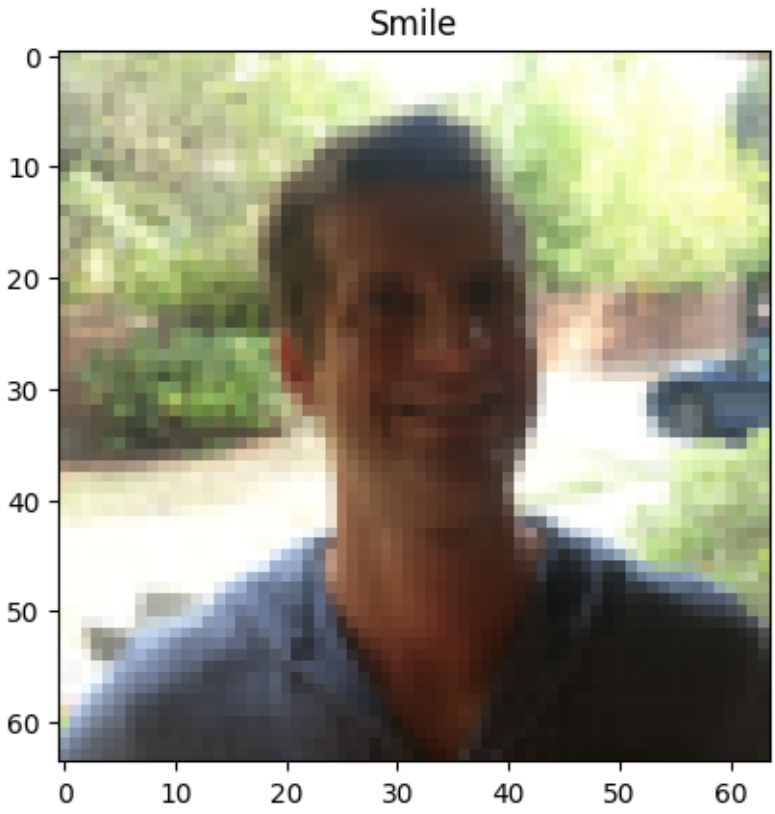
```
['list_classes', 'train_set_x', 'train_set_y']
['list_classes', 'test_set_x', 'test_set_y']
Train shape: (600, 64, 64, 3) (600,)
Test shape: (150, 64, 64, 3) (150,)
```

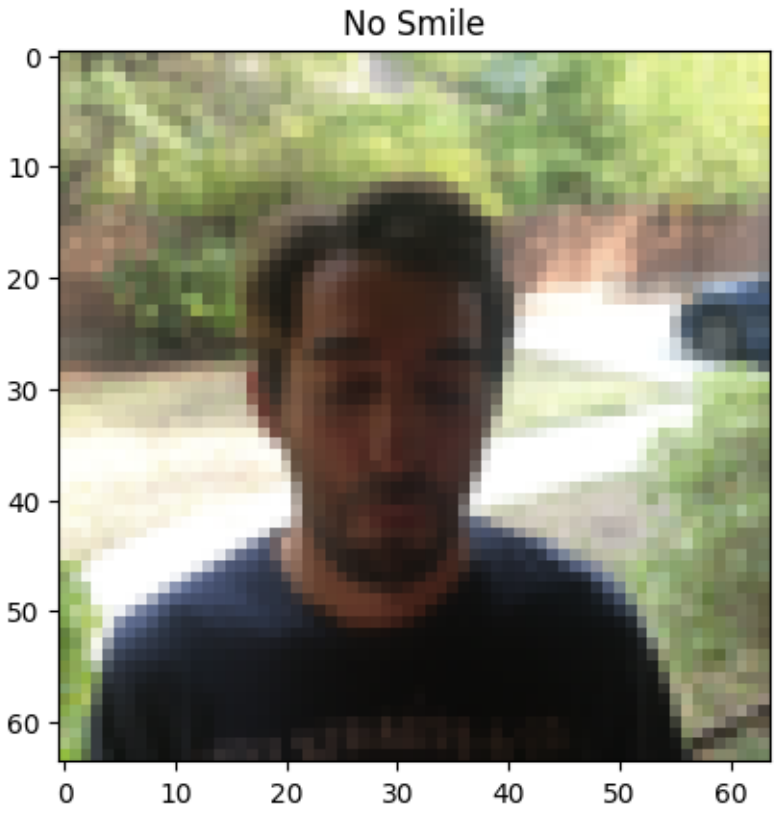
1.1 1

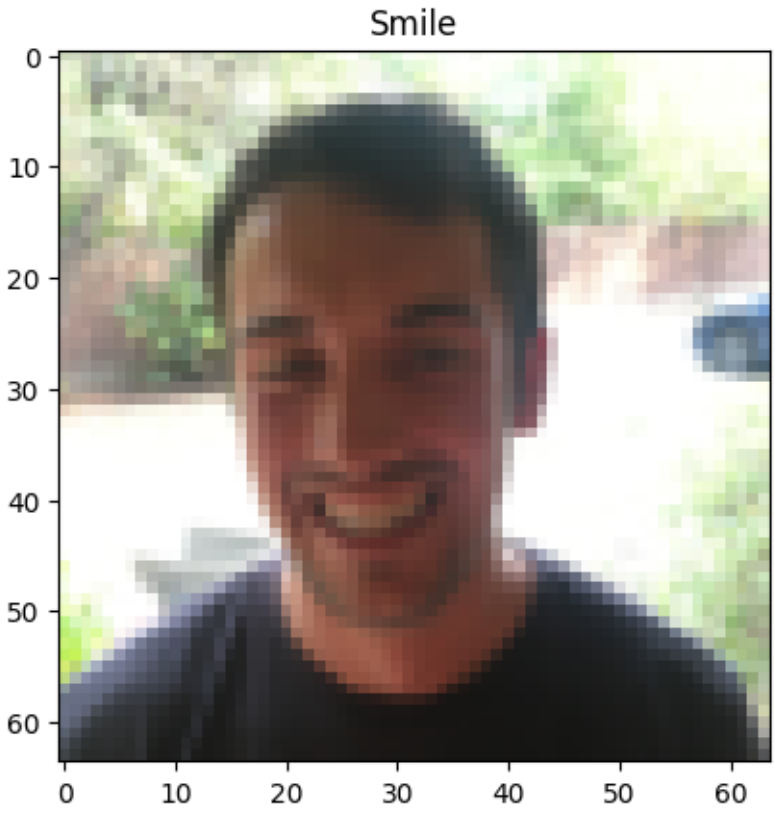
[49]:
```
for i in range(5):
    i = np.argwhere(Y_train == 0)[i][0]
    j = np.argwhere(Y_train == 1)[i][0]
    plt.imshow(X_train[i])
    plt.title("No Smile")
    plt.show()
    plt.imshow(X_train[j])
```

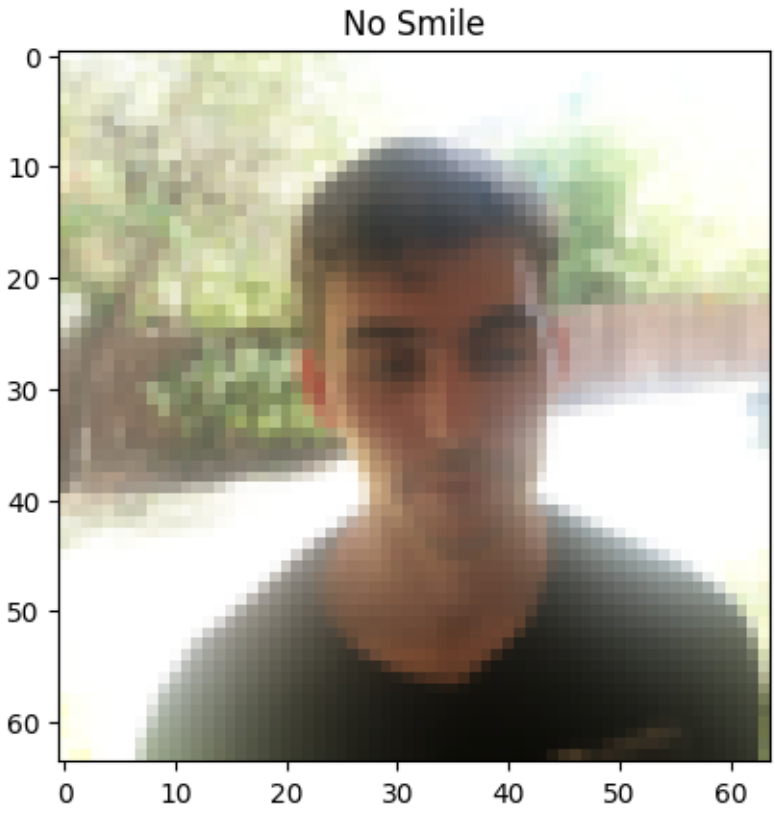
```
    plt.title("Smile")
    plt.show()
```

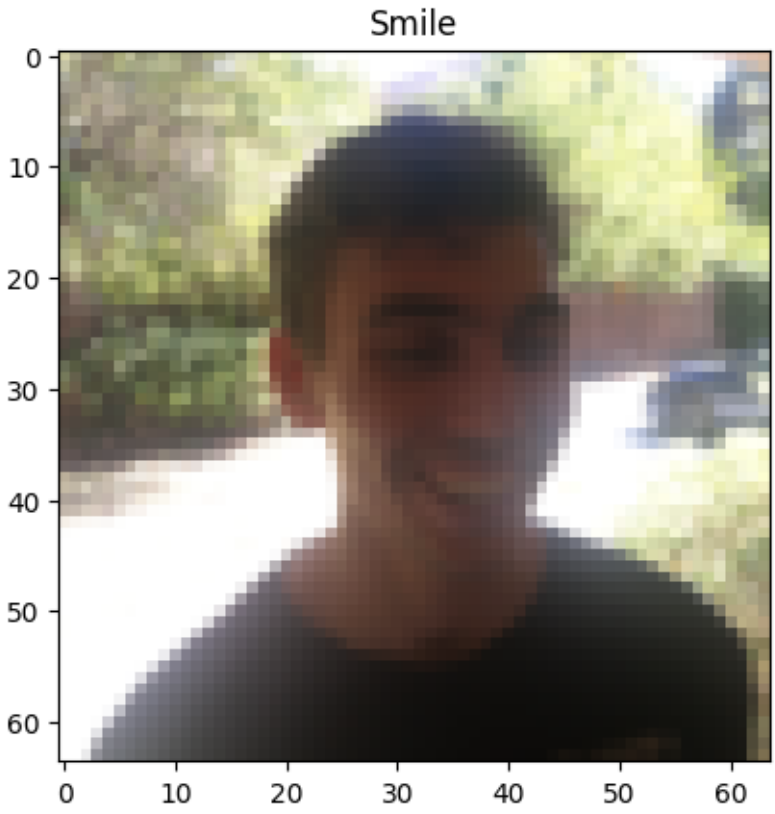


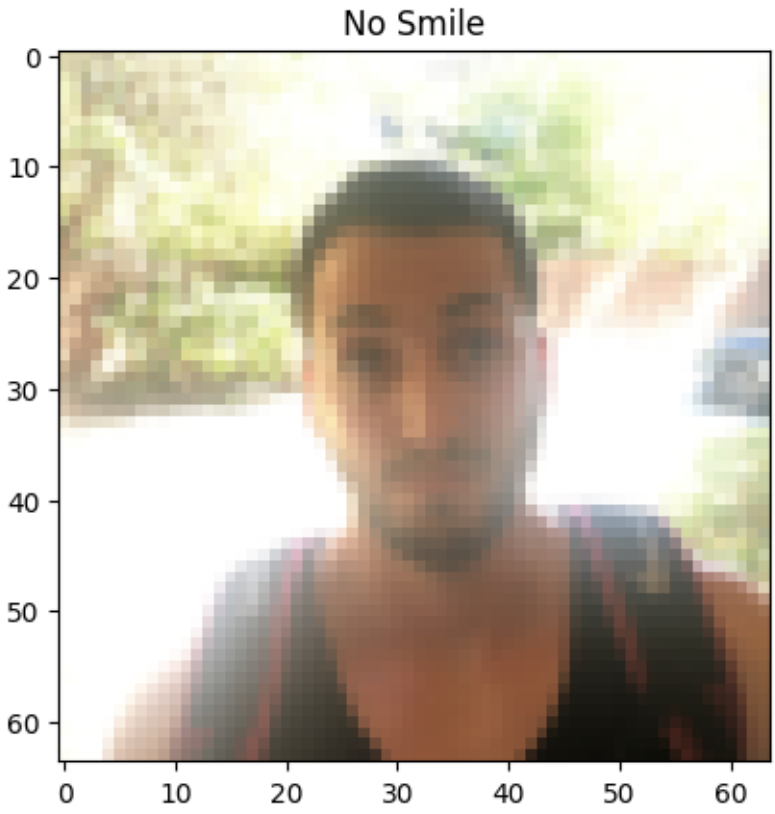

Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60

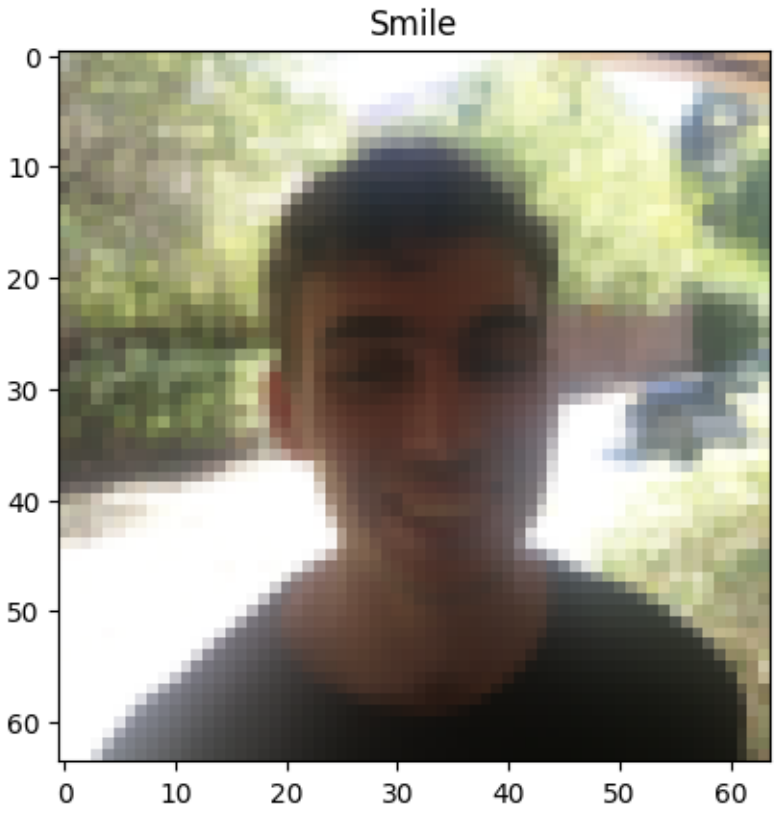

No Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60

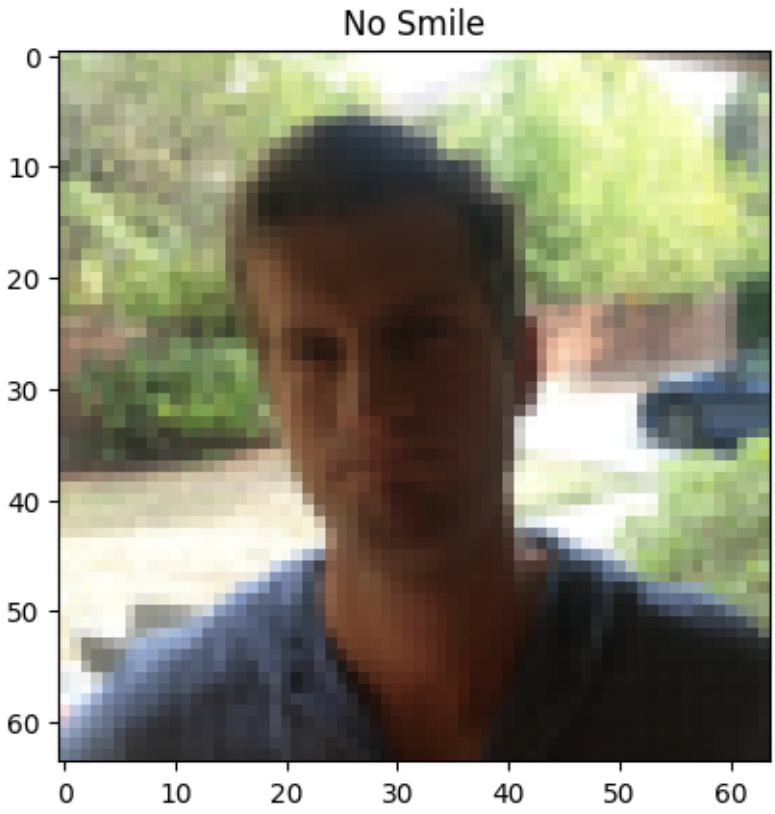

Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60

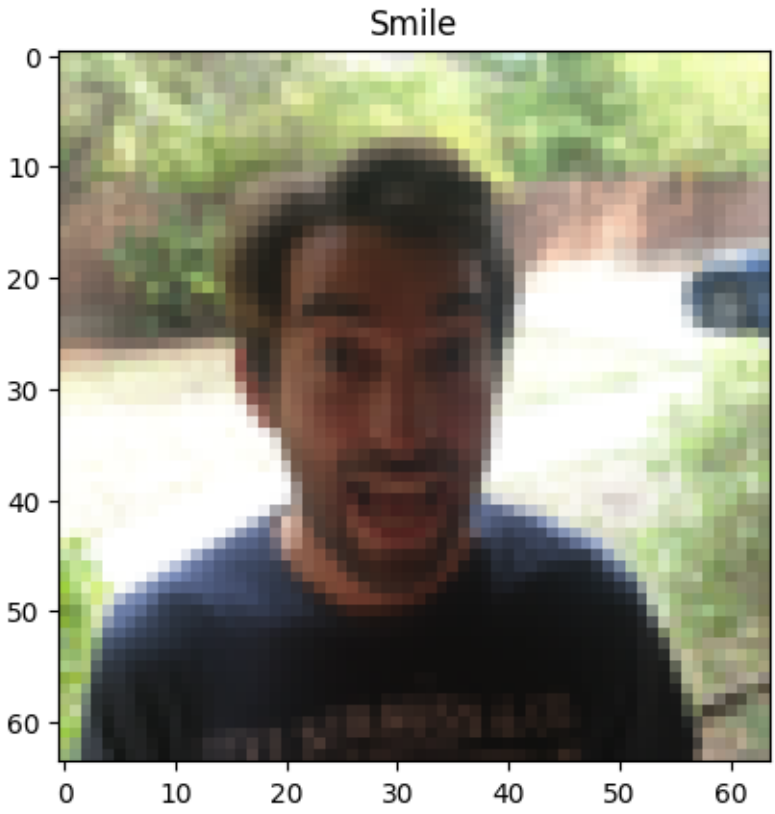

No Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60


Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60


No Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60


Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60


No Smile
0
10
20
30
40
50
60
0
10
20
30
40
50
60



1.2 2

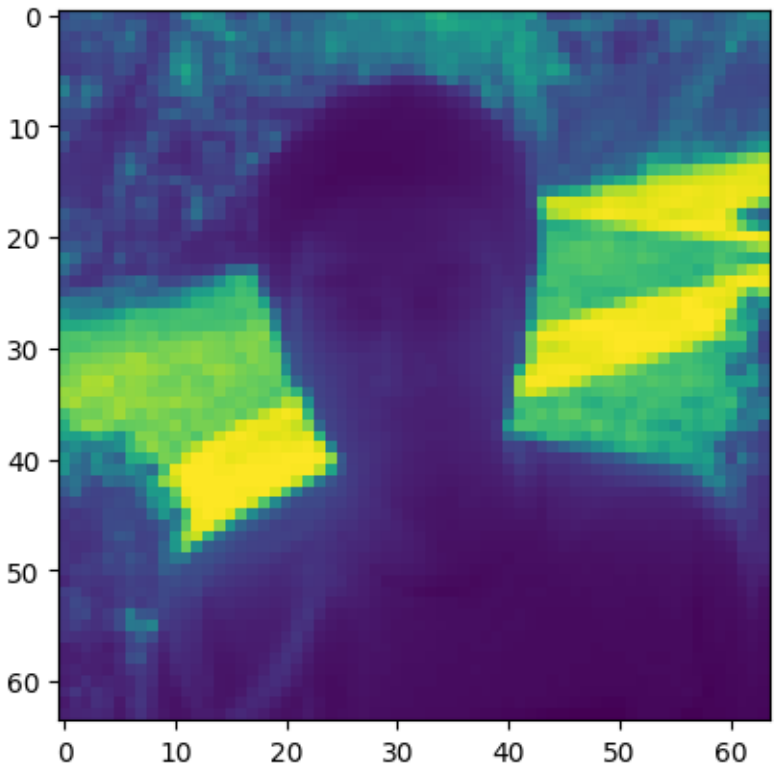
[50]:
```
smile, no_mile = np.bincount(Y_train.flatten())
print("Smile:", smile)
print("No Smile:", no_mile)
```

```
Smile: 300
No Smile: 300
```

The training dataset is perfectly balanced.

[51]:
```
# Grayscaling and Normalization
from sklearn.utils import shuffle
X_train, y_train = shuffle(X_train, Y_train)
X_train_gray_norm = np.sum(X_train/3, axis=3, keepdims=True) / 255
X_test_gray_norm = np.sum(X_test/3, axis=3, keepdims=True) / 255
```

[52]:
```
i = np.random.randint(0, len(X_train_gray_norm))
plt.imshow(X_train_gray_norm[i])
plt.show()
```



[53]:
```
# Build the model
model = Sequential()
model.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64,64,1)))
model.add(layers.MaxPooling2D(pool_size=(2,2)))

model.add(layers.Conv2D(128, (3,3), activation='relu'))
model.add(layers.AveragePooling2D(pool_size=(2,2)))


model.add(layers.Flatten())
model.add(layers.Dense(256, activation = 'relu'))
model.add(layers.Dense(2, activation = 'softmax'))
model.summary()
```

```
/opt/anaconda3/lib/python3.12/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_14"

Layer (type)	Output Shape	Param #
conv2d_38 (Conv2D)	(None, 62, 62, 64)	640
max_pooling2d_29 (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_39 (Conv2D)	(None, 29, 29, 128)	73,856
average_pooling2d_9 (AveragePooling2D)	(None, 14, 14, 128)	0
flatten_14 (Flatten)	(None, 25088)	0
dense_42 (Dense)	(None, 256)	6,422,784
dense_43 (Dense)	(None, 2)	514

Total params: 6,497,794 (24.79 MB)

Trainable params: 6,497,794 (24.79 MB)

Non-trainable params: 0 (0.00 B)

[54]:
```
# parameterizing
model.compile("Adam",  loss = 'sparse_categorical_crossentropy', metrics =␣
 ↪['accuracy'])
```

[56]:
```
# training loop
history = model.fit(X_train_gray_norm,
                    y_train,
                    batch_size = 500,
                    epochs = 20,
                    verbose = 1,
                    validation_data=(X_test_gray_norm, Y_test))
```

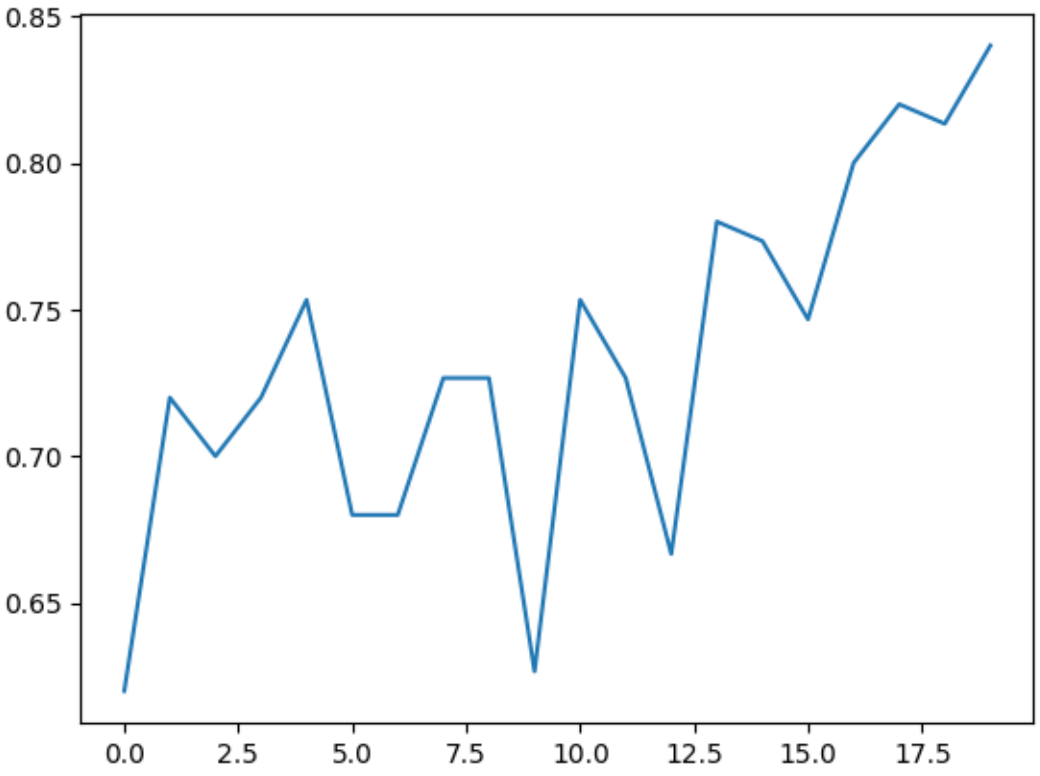
```
Epoch 1/20
2/2             1s 286ms/step -
accuracy: 0.6133 - loss: 0.6800 - val_accuracy: 0.6200 - val_loss: 0.6818
Epoch 2/20
2/2             1s 258ms/step -
accuracy: 0.6733 - loss: 0.6748 - val_accuracy: 0.7200 - val_loss: 0.6680
Epoch 3/20
2/2             1s 287ms/step -
accuracy: 0.7700 - loss: 0.6637 - val_accuracy: 0.7000 - val_loss: 0.6525
```

```
Epoch 4/20
2/2            1s 262ms/step -
accuracy: 0.6767 - loss: 0.6516 - val_accuracy: 0.7200 - val_loss: 0.6457
Epoch 5/20
2/2            1s 258ms/step -
accuracy: 0.8283 - loss: 0.6327 - val_accuracy: 0.7533 - val_loss: 0.6175
Epoch 6/20
2/2            1s 252ms/step -
accuracy: 0.7000 - loss: 0.6120 - val_accuracy: 0.6800 - val_loss: 0.6111
Epoch 7/20
2/2            1s 262ms/step -
accuracy: 0.7817 - loss: 0.5804 - val_accuracy: 0.6800 - val_loss: 0.5893
Epoch 8/20
2/2            1s 303ms/step -
accuracy: 0.7900 - loss: 0.5481 - val_accuracy: 0.7267 - val_loss: 0.5441
Epoch 9/20
2/2            1s 260ms/step -
accuracy: 0.8117 - loss: 0.5103 - val_accuracy: 0.7267 - val_loss: 0.5152
Epoch 10/20
2/2            1s 246ms/step -
accuracy: 0.7850 - loss: 0.4790 - val_accuracy: 0.6267 - val_loss: 0.6445
Epoch 11/20
2/2            1s 254ms/step -
accuracy: 0.7500 - loss: 0.5137 - val_accuracy: 0.7533 - val_loss: 0.5001
Epoch 12/20
2/2            1s 262ms/step -
accuracy: 0.7317 - loss: 0.4865 - val_accuracy: 0.7267 - val_loss: 0.5052
Epoch 13/20
2/2            1s 250ms/step -
accuracy: 0.8150 - loss: 0.4274 - val_accuracy: 0.6667 - val_loss: 0.5709
Epoch 14/20
2/2            1s 250ms/step -
accuracy: 0.7933 - loss: 0.4389 - val_accuracy: 0.7800 - val_loss: 0.4578
Epoch 15/20
2/2            1s 269ms/step -
accuracy: 0.7900 - loss: 0.4205 - val_accuracy: 0.7733 - val_loss: 0.4394
Epoch 16/20
2/2            1s 262ms/step -
accuracy: 0.8267 - loss: 0.3836 - val_accuracy: 0.7467 - val_loss: 0.4752
Epoch 17/20
2/2            1s 263ms/step -
accuracy: 0.8417 - loss: 0.3716 - val_accuracy: 0.8000 - val_loss: 0.4002
Epoch 18/20
2/2            1s 269ms/step -
accuracy: 0.8633 - loss: 0.3461 - val_accuracy: 0.8200 - val_loss: 0.3845
Epoch 19/20
2/2            1s 265ms/step -
accuracy: 0.8750 - loss: 0.3229 - val_accuracy: 0.8133 - val_loss: 0.4123
```

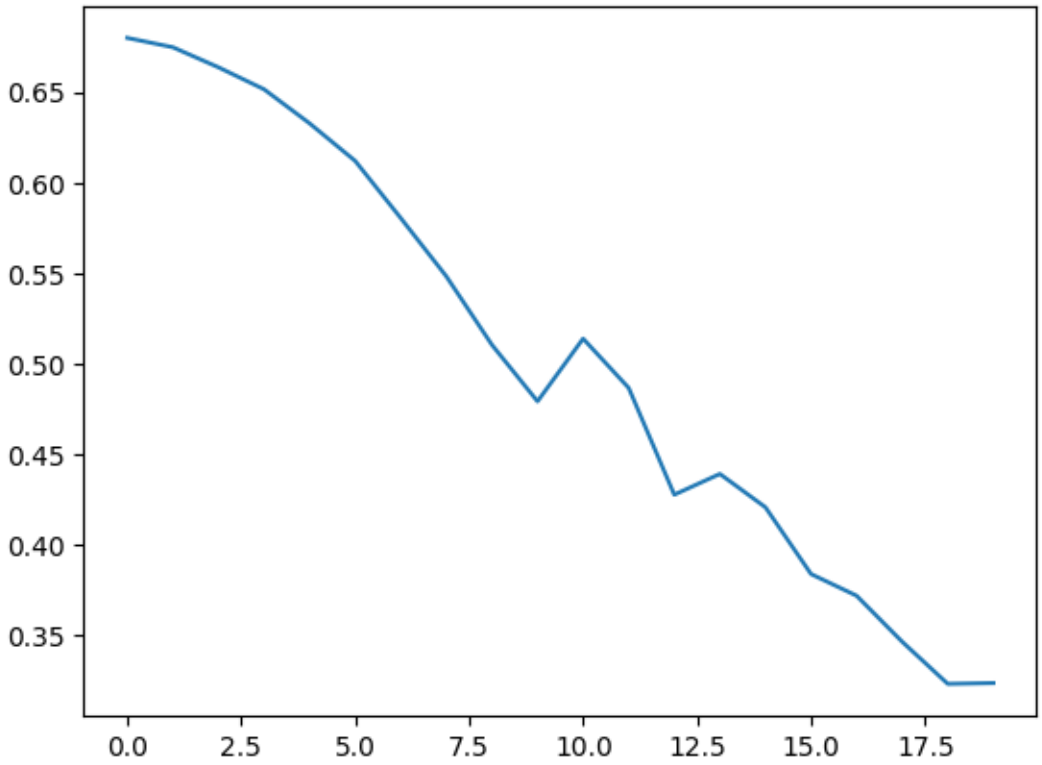
```
Epoch 20/20
2/2             1s 258ms/step -
accuracy: 0.8683 - loss: 0.3233 - val_accuracy: 0.8400 - val_loss: 0.3419
```

2 3

[57]:
```
test_accuracy = history.history['val_accuracy']
plt.plot(np.arange(0, 20), test_accuracy)
plt.show()
```



[58]:
```
test_accuracy = history.history['loss']
plt.plot(np.arange(0, 20), test_accuracy)
plt.show()
```



[60]:
```
# Test Accuracy
score = model.evaluate(X_test_gray_norm, Y_test)
print('Test Accuracy: {}'.format(score[1]))
```

```
5/5             0s 19ms/step -
accuracy: 0.8400 - loss: 0.3419
Test Accuracy: 0.8399999737739563
```

[61]:
```
# additional epochs and hidden layers and MaxPooling instead of AveragePooling
# Build the model
model1 = Sequential()
model1.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64,64,1)))
model1.add(layers.MaxPooling2D(pool_size=(2,2)))

model1.add(layers.Conv2D(128, (3,3), activation='relu'))
model1.add(layers.MaxPooling2D(pool_size=(2,2)))

model1.add(layers.Conv2D(256, (3,3), activation='relu'))
model1.add(layers.MaxPooling2D(pool_size=(2,2)))


model1.add(layers.Flatten())
```

```
model1.add(layers.Dense(256, activation = 'relu'))
model1.add(layers.Dense(2, activation = 'softmax'))
model1.summary()

# parameterizing
model1.compile("Adam",  loss = 'sparse_categorical_crossentropy', metrics =␣
 ↪['accuracy'])

# training loop
history = model1.fit(X_train_gray_norm,
                 y_train,
                 batch_size = 500,
                 epochs = 30,
                 verbose = 1,
                 validation_data=(X_test_gray_norm, Y_test))
```

```
Model: "sequential_15"
```

Layer (type)	Output Shape	Param #
conv2d_40 (Conv2D)	(None, 62, 62, 64)	640
max_pooling2d_30 (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_41 (Conv2D)	(None, 29, 29, 128)	73,856
max_pooling2d_31 (MaxPooling2D)	(None, 14, 14, 128)	0
conv2d_42 (Conv2D)	(None, 12, 12, 256)	295,168
max_pooling2d_32 (MaxPooling2D)	(None, 6, 6, 256)	0
flatten_15 (Flatten)	(None, 9216)	0
dense_44 (Dense)	(None, 256)	2,359,552
dense_45 (Dense)	(None, 2)	514

```
Total params: 2,729,730 (10.41 MB)

Trainable params: 2,729,730 (10.41 MB)

Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/30
2/2            3s 447ms/step -
accuracy: 0.5133 - loss: 0.6967 - val_accuracy: 0.5600 - val_loss: 0.8234
Epoch 2/30
2/2            2s 350ms/step -
accuracy: 0.5000 - loss: 0.8811 - val_accuracy: 0.6467 - val_loss: 0.6891
Epoch 3/30
2/2            1s 340ms/step -
accuracy: 0.5917 - loss: 0.6899 - val_accuracy: 0.4467 - val_loss: 0.6934
Epoch 4/30
2/2            1s 317ms/step -
accuracy: 0.5400 - loss: 0.6896 - val_accuracy: 0.5467 - val_loss: 0.6871
Epoch 5/30
2/2            1s 348ms/step -
accuracy: 0.4983 - loss: 0.6888 - val_accuracy: 0.5600 - val_loss: 0.6825
Epoch 6/30
2/2            1s 322ms/step -
accuracy: 0.5000 - loss: 0.6879 - val_accuracy: 0.5600 - val_loss: 0.6773
Epoch 7/30
2/2            1s 358ms/step -
accuracy: 0.5000 - loss: 0.6850 - val_accuracy: 0.5533 - val_loss: 0.6732
Epoch 8/30
2/2            2s 356ms/step -
accuracy: 0.5167 - loss: 0.6756 - val_accuracy: 0.6867 - val_loss: 0.6669
Epoch 9/30
2/2            1s 316ms/step -
accuracy: 0.7267 - loss: 0.6617 - val_accuracy: 0.6667 - val_loss: 0.6520
Epoch 10/30
2/2            1s 327ms/step -
accuracy: 0.7650 - loss: 0.6393 - val_accuracy: 0.5733 - val_loss: 0.6284
Epoch 11/30
2/2            1s 321ms/step -
accuracy: 0.5517 - loss: 0.6315 - val_accuracy: 0.6467 - val_loss: 0.6244
Epoch 12/30
2/2            1s 318ms/step -
accuracy: 0.7500 - loss: 0.5834 - val_accuracy: 0.6333 - val_loss: 0.5993
Epoch 13/30
2/2            1s 327ms/step -
accuracy: 0.7733 - loss: 0.5484 - val_accuracy: 0.6333 - val_loss: 0.6500
Epoch 14/30
2/2            1s 358ms/step -
accuracy: 0.7500 - loss: 0.5327 - val_accuracy: 0.6400 - val_loss: 0.5960
Epoch 15/30
2/2            1s 318ms/step -
accuracy: 0.6233 - loss: 0.5737 - val_accuracy: 0.6333 - val_loss: 0.6296
Epoch 16/30
2/2            1s 340ms/step -
accuracy: 0.7700 - loss: 0.4956 - val_accuracy: 0.6933 - val_loss: 0.5366
```

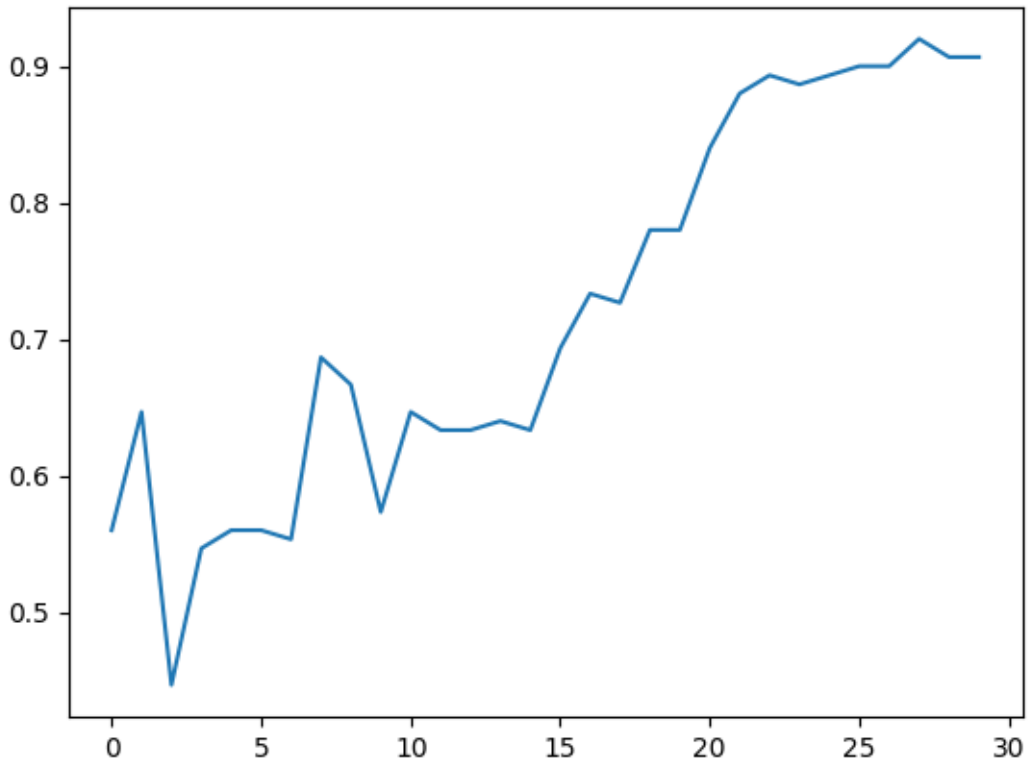
```
Epoch 17/30
2/2             2s 378ms/step -
accuracy: 0.8067 - loss: 0.4493 - val_accuracy: 0.7333 - val_loss: 0.4981
Epoch 18/30
2/2             2s 323ms/step -
accuracy: 0.8050 - loss: 0.4367 - val_accuracy: 0.7267 - val_loss: 0.5018
Epoch 19/30
2/2             1s 322ms/step -
accuracy: 0.8267 - loss: 0.3970 - val_accuracy: 0.7800 - val_loss: 0.4344
Epoch 20/30
2/2             1s 366ms/step -
accuracy: 0.8500 - loss: 0.3562 - val_accuracy: 0.7800 - val_loss: 0.4198
Epoch 21/30
2/2             1s 398ms/step -
accuracy: 0.8533 - loss: 0.3175 - val_accuracy: 0.8400 - val_loss: 0.3305
Epoch 22/30
2/2             2s 397ms/step -
accuracy: 0.8800 - loss: 0.2887 - val_accuracy: 0.8800 - val_loss: 0.2931
Epoch 23/30
2/2             2s 363ms/step -
accuracy: 0.9217 - loss: 0.2438 - val_accuracy: 0.8933 - val_loss: 0.2766
Epoch 24/30
2/2             2s 326ms/step -
accuracy: 0.9133 - loss: 0.2234 - val_accuracy: 0.8867 - val_loss: 0.2608
Epoch 25/30
2/2             1s 325ms/step -
accuracy: 0.9217 - loss: 0.2330 - val_accuracy: 0.8933 - val_loss: 0.2510
Epoch 26/30
2/2             1s 362ms/step -
accuracy: 0.9300 - loss: 0.1976 - val_accuracy: 0.9000 - val_loss: 0.2461
Epoch 27/30
2/2             1s 352ms/step -
accuracy: 0.9250 - loss: 0.1844 - val_accuracy: 0.9000 - val_loss: 0.2102
Epoch 28/30
2/2             1s 325ms/step -
accuracy: 0.9383 - loss: 0.1776 - val_accuracy: 0.9200 - val_loss: 0.2205
Epoch 29/30
2/2             1s 362ms/step -
accuracy: 0.9383 - loss: 0.1621 - val_accuracy: 0.9067 - val_loss: 0.2318
Epoch 30/30
2/2             1s 337ms/step -
accuracy: 0.9383 - loss: 0.1538 - val_accuracy: 0.9067 - val_loss: 0.2028
```

[62]:
```
# Test Accuracy
score = model1.evaluate(X_test_gray_norm, Y_test)
print('Test Accuracy: {}'.format(score[1]))
```

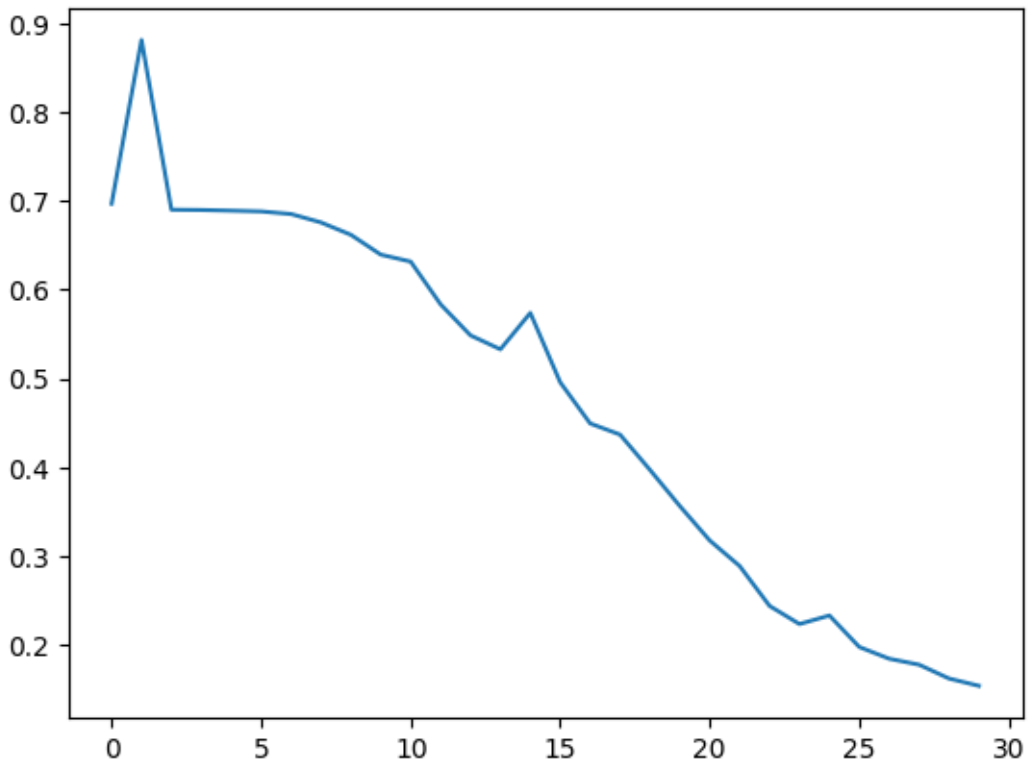
```
5/5             0s 22ms/step -
```

```
accuracy: 0.9067 - loss: 0.2028
Test Accuracy: 0.9066666960716248
```

[64]:
```
test_accuracy = history.history['val_accuracy']
plt.plot(np.arange(0, 30), test_accuracy)
plt.show()
```



[66]:
```
test_accuracy = history.history['loss']
plt.plot(np.arange(0, 30), test_accuracy)
plt.show()
```



[67]:
```
# additional epochs and hidden layers + dropout
# and MaxPooling instead of AveragePooling + lower batch size
# Build the model
model2 = Sequential()
model2.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64,64,1)))
model2.add(layers.MaxPooling2D(pool_size=(2,2)))

model2.add(layers.Conv2D(128, (3,3), activation='relu'))
model2.add(layers.MaxPooling2D(pool_size=(2,2)))

model2.add(layers.Conv2D(256, (3,3), activation='relu'))
model2.add(layers.MaxPooling2D(pool_size=(2,2)))


model2.add(layers.Flatten())
model2.add(layers.Dense(256, activation = 'relu'))
model2.add(layers.Dense(128, activation = 'relu'))
model2.add(layers.Dropout(0.5))
model2.add(layers.Dense(2, activation = 'softmax'))
model2.summary()

# parameterizing
```

```
model2.compile("Adam",  loss = 'sparse_categorical_crossentropy', metrics =␣
 ↪['accuracy'])

# training loop
history = model2.fit(X_train_gray_norm,
                 y_train,
                 batch_size = 300,
                 epochs = 30,
                 verbose = 1,
                 validation_data=(X_test_gray_norm, Y_test))
```

```
Model: "sequential_16"
```

Layer (type)	Output Shape	Param #
conv2d_43 (Conv2D)	(None, 62, 62, 64)	640
max_pooling2d_33 (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_44 (Conv2D)	(None, 29, 29, 128)	73,856
max_pooling2d_34 (MaxPooling2D)	(None, 14, 14, 128)	0
conv2d_45 (Conv2D)	(None, 12, 12, 256)	295,168
max_pooling2d_35 (MaxPooling2D)	(None, 6, 6, 256)	0
flatten_16 (Flatten)	(None, 9216)	0
dense_46 (Dense)	(None, 256)	2,359,552
dense_47 (Dense)	(None, 128)	32,896
dropout_3 (Dropout)	(None, 128)	0
dense_48 (Dense)	(None, 2)	258

```
Total params: 2,762,370 (10.54 MB)

Trainable params: 2,762,370 (10.54 MB)

Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/30
2/2            3s 949ms/step -
accuracy: 0.5133 - loss: 0.7031 - val_accuracy: 0.4400 - val_loss: 0.7352
Epoch 2/30
2/2            1s 787ms/step -
accuracy: 0.5283 - loss: 0.7031 - val_accuracy: 0.5600 - val_loss: 0.6877
Epoch 3/30
2/2            1s 809ms/step -
accuracy: 0.4917 - loss: 0.6933 - val_accuracy: 0.5600 - val_loss: 0.6879
Epoch 4/30
2/2            1s 743ms/step -
accuracy: 0.5550 - loss: 0.6892 - val_accuracy: 0.5600 - val_loss: 0.6876
Epoch 5/30
2/2            1s 700ms/step -
accuracy: 0.5517 - loss: 0.6887 - val_accuracy: 0.5333 - val_loss: 0.6842
Epoch 6/30
2/2            1s 701ms/step -
accuracy: 0.5750 - loss: 0.6847 - val_accuracy: 0.5733 - val_loss: 0.6781
Epoch 7/30
2/2            1s 704ms/step -
accuracy: 0.5833 - loss: 0.6789 - val_accuracy: 0.6733 - val_loss: 0.6736
Epoch 8/30
2/2            1s 723ms/step -
accuracy: 0.6033 - loss: 0.6681 - val_accuracy: 0.6333 - val_loss: 0.6708
Epoch 9/30
2/2            1s 682ms/step -
accuracy: 0.6333 - loss: 0.6568 - val_accuracy: 0.6400 - val_loss: 0.6367
Epoch 10/30
2/2            1s 720ms/step -
accuracy: 0.6767 - loss: 0.6277 - val_accuracy: 0.6333 - val_loss: 0.6310
Epoch 11/30
2/2            1s 831ms/step -
accuracy: 0.7417 - loss: 0.5843 - val_accuracy: 0.6933 - val_loss: 0.5877
Epoch 12/30
2/2            1s 729ms/step -
accuracy: 0.6983 - loss: 0.5947 - val_accuracy: 0.6933 - val_loss: 0.5785
Epoch 13/30
2/2            1s 753ms/step -
accuracy: 0.7350 - loss: 0.5281 - val_accuracy: 0.6667 - val_loss: 0.5757
Epoch 14/30
2/2            1s 699ms/step -
accuracy: 0.7917 - loss: 0.5002 - val_accuracy: 0.7133 - val_loss: 0.5331
Epoch 15/30
2/2            1s 696ms/step -
accuracy: 0.7933 - loss: 0.4524 - val_accuracy: 0.7333 - val_loss: 0.5123
Epoch 16/30
2/2            1s 745ms/step -
accuracy: 0.8117 - loss: 0.4240 - val_accuracy: 0.7133 - val_loss: 0.5206
```

```
Epoch 17/30
2/2             1s 751ms/step -
accuracy: 0.8233 - loss: 0.4065 - val_accuracy: 0.7867 - val_loss: 0.4281
Epoch 18/30
2/2             1s 729ms/step -
accuracy: 0.8183 - loss: 0.3807 - val_accuracy: 0.7467 - val_loss: 0.4891
Epoch 19/30
2/2             1s 744ms/step -
accuracy: 0.8517 - loss: 0.3571 - val_accuracy: 0.8333 - val_loss: 0.3576
Epoch 20/30
2/2             1s 754ms/step -
accuracy: 0.8367 - loss: 0.3750 - val_accuracy: 0.8667 - val_loss: 0.3301
Epoch 21/30
2/2             1s 721ms/step -
accuracy: 0.8883 - loss: 0.2977 - val_accuracy: 0.8067 - val_loss: 0.3884
Epoch 22/30
2/2             1s 780ms/step -
accuracy: 0.8833 - loss: 0.2856 - val_accuracy: 0.8733 - val_loss: 0.2838
Epoch 23/30
2/2             1s 773ms/step -
accuracy: 0.9167 - loss: 0.2652 - val_accuracy: 0.8733 - val_loss: 0.2850
Epoch 24/30
2/2             1s 695ms/step -
accuracy: 0.8900 - loss: 0.2655 - val_accuracy: 0.8933 - val_loss: 0.2305
Epoch 25/30
2/2             1s 763ms/step -
accuracy: 0.9283 - loss: 0.2166 - val_accuracy: 0.8933 - val_loss: 0.2218
Epoch 26/30
2/2             1s 696ms/step -
accuracy: 0.9250 - loss: 0.2015 - val_accuracy: 0.8867 - val_loss: 0.2511
Epoch 27/30
2/2             1s 712ms/step -
accuracy: 0.9400 - loss: 0.1780 - val_accuracy: 0.9000 - val_loss: 0.2264
Epoch 28/30
2/2             1s 724ms/step -
accuracy: 0.9467 - loss: 0.1679 - val_accuracy: 0.9200 - val_loss: 0.2116
Epoch 29/30
2/2             1s 720ms/step -
accuracy: 0.9517 - loss: 0.1480 - val_accuracy: 0.9200 - val_loss: 0.1924
Epoch 30/30
2/2             1s 758ms/step -
accuracy: 0.9533 - loss: 0.1359 - val_accuracy: 0.9200 - val_loss: 0.1744
```

[68]:
```
# Test Accuracy
score = model2.evaluate(X_test_gray_norm, Y_test)
print('Test Accuracy: {}'.format(score[1]))
```

```
5/5             0s 23ms/step -
```

```
accuracy: 0.9200 - loss: 0.1744
Test Accuracy: 0.9200000166893005
```

3 4

[69]:
```
# additional epochs and hidden layers + dropout
# and MaxPooling instead of AveragePooling + lower batch size
# Build the model
model2 = Sequential()
model2.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64,64,1)))
model2.add(layers.MaxPooling2D(pool_size=(2,2)))

model2.add(layers.Conv2D(128, (3,3), activation='relu'))
model2.add(layers.MaxPooling2D(pool_size=(2,2)))

model2.add(layers.Conv2D(256, (3,3), activation='relu'))
model2.add(layers.MaxPooling2D(pool_size=(2,2)))


model2.add(layers.Flatten())
model2.add(layers.Dense(256, activation = 'relu'))
model2.add(layers.Dense(128, activation = 'relu'))
model2.add(layers.Dropout(0.5))
model2.add(layers.Dense(2, activation = 'softmax'))
model2.summary()

# parameterizing
model2.compile("Adam",  loss = 'sparse_categorical_crossentropy', metrics =␣
 ↪['accuracy'])

# training loop
history = model2.fit(X_train_gray_norm,
                 y_train,
                 batch_size = 300,
                 epochs = 30,
                 verbose = 1,
                 validation_data=(X_test_gray_norm, Y_test))
```

Model: "sequential_17"

Layer (type)	Output Shape	Param #
conv2d_46 (Conv2D)	(None, 62, 62, 64)	640
max_pooling2d_36 (MaxPooling2D)	(None, 31, 31, 64)	0

conv2d_47 (Conv2D)	(None, 29, 29, 128)	73,856
max_pooling2d_37 (MaxPooling2D)	(None, 14, 14, 128)	0
conv2d_48 (Conv2D)	(None, 12, 12, 256)	295,168
max_pooling2d_38 (MaxPooling2D)	(None, 6, 6, 256)	0
flatten_17 (Flatten)	(None, 9216)	0
dense_49 (Dense)	(None, 256)	2,359,552
dense_50 (Dense)	(None, 128)	32,896
dropout_4 (Dropout)	(None, 128)	0
dense_51 (Dense)	(None, 2)	258

Total params: 2,762,370 (10.54 MB)

Trainable params: 2,762,370 (10.54 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/30
2/2             2s 924ms/step -
accuracy: 0.5267 - loss: 0.6921 - val_accuracy: 0.4400 - val_loss: 0.8206
Epoch 2/30
2/2             1s 799ms/step -
accuracy: 0.5250 - loss: 0.7312 - val_accuracy: 0.5600 - val_loss: 0.6800
Epoch 3/30
2/2             1s 696ms/step -
accuracy: 0.5117 - loss: 0.6954 - val_accuracy: 0.5600 - val_loss: 0.6878
Epoch 4/30
2/2             1s 695ms/step -
accuracy: 0.5333 - loss: 0.6902 - val_accuracy: 0.5933 - val_loss: 0.6859
Epoch 5/30
2/2             1s 691ms/step -
accuracy: 0.5733 - loss: 0.6869 - val_accuracy: 0.5933 - val_loss: 0.6797
Epoch 6/30
2/2             1s 695ms/step -
accuracy: 0.6150 - loss: 0.6771 - val_accuracy: 0.5867 - val_loss: 0.6671
Epoch 7/30
2/2             1s 703ms/step -
accuracy: 0.6533 - loss: 0.6612 - val_accuracy: 0.7200 - val_loss: 0.6530
```

```
Epoch 8/30
2/2             1s 734ms/step -
accuracy: 0.6900 - loss: 0.6382 - val_accuracy: 0.6533 - val_loss: 0.6318
Epoch 9/30
2/2             1s 725ms/step -
accuracy: 0.7167 - loss: 0.5892 - val_accuracy: 0.6200 - val_loss: 0.6671
Epoch 10/30
2/2             1s 692ms/step -
accuracy: 0.5933 - loss: 0.6526 - val_accuracy: 0.6667 - val_loss: 0.5890
Epoch 11/30
2/2             1s 767ms/step -
accuracy: 0.7617 - loss: 0.5574 - val_accuracy: 0.6400 - val_loss: 0.6211
Epoch 12/30
2/2             1s 740ms/step -
accuracy: 0.7783 - loss: 0.5283 - val_accuracy: 0.7200 - val_loss: 0.5515
Epoch 13/30
2/2             1s 723ms/step -
accuracy: 0.7083 - loss: 0.5309 - val_accuracy: 0.7333 - val_loss: 0.5430
Epoch 14/30
2/2             1s 676ms/step -
accuracy: 0.8267 - loss: 0.4651 - val_accuracy: 0.7400 - val_loss: 0.5076
Epoch 15/30
2/2             1s 710ms/step -
accuracy: 0.8100 - loss: 0.4355 - val_accuracy: 0.7533 - val_loss: 0.4805
Epoch 16/30
2/2             2s 889ms/step -
accuracy: 0.8433 - loss: 0.3861 - val_accuracy: 0.8000 - val_loss: 0.4084
Epoch 17/30
2/2             1s 709ms/step -
accuracy: 0.8500 - loss: 0.3411 - val_accuracy: 0.8267 - val_loss: 0.3916
Epoch 18/30
2/2             1s 708ms/step -
accuracy: 0.7917 - loss: 0.4309 - val_accuracy: 0.7267 - val_loss: 0.4601
Epoch 19/30
2/2             1s 871ms/step -
accuracy: 0.8683 - loss: 0.3441 - val_accuracy: 0.8533 - val_loss: 0.3647
Epoch 20/30
2/2             1s 747ms/step -
accuracy: 0.8733 - loss: 0.3412 - val_accuracy: 0.8667 - val_loss: 0.3482
Epoch 21/30
2/2             1s 729ms/step -
accuracy: 0.8950 - loss: 0.2784 - val_accuracy: 0.8800 - val_loss: 0.3047
Epoch 22/30
2/2             1s 722ms/step -
accuracy: 0.9183 - loss: 0.2366 - val_accuracy: 0.8800 - val_loss: 0.2660
Epoch 23/30
2/2             1s 710ms/step -
accuracy: 0.9117 - loss: 0.2327 - val_accuracy: 0.8733 - val_loss: 0.2947
```

```
Epoch 24/30
2/2             1s 690ms/step -
accuracy: 0.9167 - loss: 0.2122 - val_accuracy: 0.8867 - val_loss: 0.2122
Epoch 25/30
2/2             1s 691ms/step -
accuracy: 0.9417 - loss: 0.1996 - val_accuracy: 0.9200 - val_loss: 0.2164
Epoch 26/30
2/2             1s 697ms/step -
accuracy: 0.9100 - loss: 0.2053 - val_accuracy: 0.9067 - val_loss: 0.2006
Epoch 27/30
2/2             1s 750ms/step -
accuracy: 0.9167 - loss: 0.2361 - val_accuracy: 0.9067 - val_loss: 0.2055
Epoch 28/30
2/2             1s 724ms/step -
accuracy: 0.9417 - loss: 0.1688 - val_accuracy: 0.8733 - val_loss: 0.2998
Epoch 29/30
2/2             1s 730ms/step -
accuracy: 0.9283 - loss: 0.1793 - val_accuracy: 0.9200 - val_loss: 0.1909
Epoch 30/30
2/2             1s 755ms/step -
accuracy: 0.9567 - loss: 0.1618 - val_accuracy: 0.9267 - val_loss: 0.1934
```

[70]:
```
# Number of hidden layers and neurons increased and activation function changed␣
  ↪to tanh
model3 = Sequential()

model3.add(layers.Conv2D(64, (3,3), activation='relu', input_shape=(64,64,1)))
model3.add(layers.MaxPooling2D((2,2)))

model3.add(layers.Conv2D(128, (3,3), activation='relu'))
model3.add(layers.MaxPooling2D((2,2)))

model3.add(layers.Conv2D(256, (3,3), activation='relu'))
model3.add(layers.MaxPooling2D((2,2)))

model3.add(layers.Flatten())
model3.add(layers.Dense(256, activation='tanh'))
model3.add(layers.Dense(128, activation='tanh'))
model3.add(layers.Dense(64, activation='tanh'))
model3.add(layers.Dense(2, activation='softmax'))

model3.compile("Adam", loss='sparse_categorical_crossentropy',␣
  ↪metrics=['accuracy'])

history3 = model3.fit(
    X_train_gray_norm,
    y_train,
```

```
    batch_size=300,
    epochs=30,
    validation_data=(X_test_gray_norm, Y_test)
)

# Test Accuracy
score = model3.evaluate(X_test_gray_norm, Y_test)
print('Test Accuracy: {}'.format(score[1]))
```

```
Epoch 1/30
2/2             3s 968ms/step -
accuracy: 0.5000 - loss: 1.7149 - val_accuracy: 0.4400 - val_loss: 0.8496
Epoch 2/30
2/2             2s 948ms/step -
accuracy: 0.5100 - loss: 1.0023 - val_accuracy: 0.5600 - val_loss: 0.7461
Epoch 3/30
2/2             2s 795ms/step -
accuracy: 0.5000 - loss: 0.7369 - val_accuracy: 0.4400 - val_loss: 0.7016
Epoch 4/30
2/2             2s 905ms/step -
accuracy: 0.5000 - loss: 0.6956 - val_accuracy: 0.4400 - val_loss: 0.7121
Epoch 5/30
2/2             1s 766ms/step -
accuracy: 0.5000 - loss: 0.6974 - val_accuracy: 0.4400 - val_loss: 0.7004
Epoch 6/30
2/2             1s 818ms/step -
accuracy: 0.5000 - loss: 0.6949 - val_accuracy: 0.4533 - val_loss: 0.6934
Epoch 7/30
2/2             2s 893ms/step -
accuracy: 0.5233 - loss: 0.6925 - val_accuracy: 0.5533 - val_loss: 0.6915
Epoch 8/30
2/2             1s 741ms/step -
accuracy: 0.5467 - loss: 0.6921 - val_accuracy: 0.6000 - val_loss: 0.6914
Epoch 9/30
2/2             1s 747ms/step -
accuracy: 0.6017 - loss: 0.6915 - val_accuracy: 0.6267 - val_loss: 0.6909
Epoch 10/30
2/2             1s 780ms/step -
accuracy: 0.6383 - loss: 0.6901 - val_accuracy: 0.6000 - val_loss: 0.6884
Epoch 11/30
2/2             1s 830ms/step -
accuracy: 0.5567 - loss: 0.6882 - val_accuracy: 0.6800 - val_loss: 0.6875
Epoch 12/30
2/2             1s 723ms/step -
accuracy: 0.6950 - loss: 0.6848 - val_accuracy: 0.6733 - val_loss: 0.6847
Epoch 13/30
2/2             1s 721ms/step -
accuracy: 0.6633 - loss: 0.6798 - val_accuracy: 0.6800 - val_loss: 0.6802
```

```
Epoch 14/30
2/2            1s 676ms/step -
accuracy: 0.6067 - loss: 0.6759 - val_accuracy: 0.6467 - val_loss: 0.6734
Epoch 15/30
2/2            1s 697ms/step -
accuracy: 0.5850 - loss: 0.6736 - val_accuracy: 0.6667 - val_loss: 0.6629
Epoch 16/30
2/2            1s 685ms/step -
accuracy: 0.6283 - loss: 0.6600 - val_accuracy: 0.5133 - val_loss: 0.6865
Epoch 17/30
2/2            1s 790ms/step -
accuracy: 0.6833 - loss: 0.6444 - val_accuracy: 0.5533 - val_loss: 0.6443
Epoch 18/30
2/2            1s 763ms/step -
accuracy: 0.5683 - loss: 0.6369 - val_accuracy: 0.6133 - val_loss: 0.6673
Epoch 19/30
2/2            2s 903ms/step -
accuracy: 0.6950 - loss: 0.6202 - val_accuracy: 0.6667 - val_loss: 0.6244
Epoch 20/30
2/2            2s 995ms/step -
accuracy: 0.7200 - loss: 0.5838 - val_accuracy: 0.6733 - val_loss: 0.6128
Epoch 21/30
2/2            1s 758ms/step -
accuracy: 0.7533 - loss: 0.5584 - val_accuracy: 0.6800 - val_loss: 0.5893
Epoch 22/30
2/2            2s 844ms/step -
accuracy: 0.7500 - loss: 0.5248 - val_accuracy: 0.6667 - val_loss: 0.5786
Epoch 23/30
2/2            2s 796ms/step -
accuracy: 0.7850 - loss: 0.4910 - val_accuracy: 0.7200 - val_loss: 0.5519
Epoch 24/30
2/2            1s 804ms/step -
accuracy: 0.7267 - loss: 0.5250 - val_accuracy: 0.6800 - val_loss: 0.5627
Epoch 25/30
2/2            2s 793ms/step -
accuracy: 0.7517 - loss: 0.4651 - val_accuracy: 0.6867 - val_loss: 0.5604
Epoch 26/30
2/2            1s 741ms/step -
accuracy: 0.7917 - loss: 0.4352 - val_accuracy: 0.7267 - val_loss: 0.5050
Epoch 27/30
2/2            2s 945ms/step -
accuracy: 0.7567 - loss: 0.4477 - val_accuracy: 0.7267 - val_loss: 0.5061
Epoch 28/30
2/2            1s 755ms/step -
accuracy: 0.8117 - loss: 0.3988 - val_accuracy: 0.7333 - val_loss: 0.4895
Epoch 29/30
2/2            2s 800ms/step -
accuracy: 0.8200 - loss: 0.3766 - val_accuracy: 0.8133 - val_loss: 0.4083
```

```
Epoch 30/30
2/2             1s 808ms/step -
accuracy: 0.8533 - loss: 0.3277 - val_accuracy: 0.7467 - val_loss: 0.4813
5/5             0s 24ms/step -
accuracy: 0.7467 - loss: 0.4813
Test Accuracy: 0.746666669845581
```

[71]:
```
# Number of hidden layers and neurons increased and activation function changed␣
↪to tanh for both convolutional and dense layers
model4 = Sequential()

model4.add(layers.Conv2D(64, (3,3), activation='tanh', input_shape=(64,64,1)))
model4.add(layers.MaxPooling2D((2,2)))

model4.add(layers.Conv2D(128, (3,3), activation='tanh'))
model4.add(layers.MaxPooling2D((2,2)))

model4.add(layers.Conv2D(256, (3,3), activation='tanh'))
model4.add(layers.MaxPooling2D((2,2)))

model4.add(layers.Flatten())
model4.add(layers.Dense(256, activation='tanh'))
model4.add(layers.Dense(128, activation='tanh'))
model4.add(layers.Dense(64, activation='relu'))
model4.add(layers.Dense(32, activation='relu'))
model4.add(layers.Dense(2, activation='softmax'))

model4.compile("Adam", loss='sparse_categorical_crossentropy',␣
↪metrics=['accuracy'])

history4 = model4.fit(
    X_train_gray_norm,
    y_train,
    batch_size=300,
    epochs=30,
    validation_data=(X_test_gray_norm, Y_test)
)

# Test Accuracy
score = model4.evaluate(X_test_gray_norm, Y_test)
print('Test Accuracy: {}'.format(score[1]))
```

```
Epoch 1/30
2/2             3s 891ms/step -
accuracy: 0.6217 - loss: 0.6705 - val_accuracy: 0.5600 - val_loss: 0.9163
Epoch 2/30
2/2             1s 751ms/step -
accuracy: 0.5067 - loss: 1.0357 - val_accuracy: 0.4400 - val_loss: 0.7431
```

```
Epoch 3/30
2/2             1s 726ms/step -
accuracy: 0.5267 - loss: 0.6939 - val_accuracy: 0.5600 - val_loss: 0.6782
Epoch 4/30
2/2             2s 866ms/step -
accuracy: 0.5033 - loss: 0.6878 - val_accuracy: 0.6133 - val_loss: 0.6830
Epoch 5/30
2/2             1s 786ms/step -
accuracy: 0.5983 - loss: 0.7093 - val_accuracy: 0.5267 - val_loss: 0.6913
Epoch 6/30
2/2             1s 833ms/step -
accuracy: 0.5583 - loss: 0.6764 - val_accuracy: 0.4867 - val_loss: 0.6967
Epoch 7/30
2/2             2s 892ms/step -
accuracy: 0.5233 - loss: 0.6702 - val_accuracy: 0.5467 - val_loss: 0.6695
Epoch 8/30
2/2             1s 829ms/step -
accuracy: 0.5600 - loss: 0.6666 - val_accuracy: 0.5067 - val_loss: 0.7041
Epoch 9/30
2/2             1s 731ms/step -
accuracy: 0.6367 - loss: 0.6434 - val_accuracy: 0.6133 - val_loss: 0.6470
Epoch 10/30
2/2             1s 850ms/step -
accuracy: 0.6800 - loss: 0.6218 - val_accuracy: 0.6067 - val_loss: 0.6555
Epoch 11/30
2/2             1s 794ms/step -
accuracy: 0.6850 - loss: 0.6028 - val_accuracy: 0.6200 - val_loss: 0.6468
Epoch 12/30
2/2             1s 812ms/step -
accuracy: 0.7217 - loss: 0.5744 - val_accuracy: 0.6800 - val_loss: 0.6070
Epoch 13/30
2/2             1s 733ms/step -
accuracy: 0.7067 - loss: 0.5630 - val_accuracy: 0.6600 - val_loss: 0.6137
Epoch 14/30
2/2             1s 711ms/step -
accuracy: 0.7633 - loss: 0.5210 - val_accuracy: 0.6733 - val_loss: 0.5615
Epoch 15/30
2/2             1s 741ms/step -
accuracy: 0.7700 - loss: 0.4829 - val_accuracy: 0.6933 - val_loss: 0.5580
Epoch 16/30
2/2             1s 720ms/step -
accuracy: 0.8050 - loss: 0.4447 - val_accuracy: 0.6933 - val_loss: 0.5494
Epoch 17/30
2/2             1s 795ms/step -
accuracy: 0.7950 - loss: 0.4391 - val_accuracy: 0.6867 - val_loss: 0.5521
Epoch 18/30
2/2             1s 814ms/step -
accuracy: 0.8183 - loss: 0.3929 - val_accuracy: 0.8267 - val_loss: 0.4161
```

```
Epoch 19/30
2/2             1s 790ms/step -
accuracy: 0.8383 - loss: 0.3930 - val_accuracy: 0.8133 - val_loss: 0.4065
Epoch 20/30
2/2             1s 742ms/step -
accuracy: 0.8400 - loss: 0.3379 - val_accuracy: 0.7667 - val_loss: 0.4319
Epoch 21/30
2/2             1s 776ms/step -
accuracy: 0.8733 - loss: 0.3112 - val_accuracy: 0.8067 - val_loss: 0.3819
Epoch 22/30
2/2             1s 767ms/step -
accuracy: 0.8083 - loss: 0.4011 - val_accuracy: 0.8667 - val_loss: 0.3210
Epoch 23/30
2/2             1s 731ms/step -
accuracy: 0.8033 - loss: 0.3755 - val_accuracy: 0.8467 - val_loss: 0.3585
Epoch 24/30
2/2             1s 752ms/step -
accuracy: 0.8533 - loss: 0.3184 - val_accuracy: 0.8533 - val_loss: 0.2938
Epoch 25/30
2/2             1s 818ms/step -
accuracy: 0.8817 - loss: 0.3006 - val_accuracy: 0.8467 - val_loss: 0.3296
Epoch 26/30
2/2             1s 733ms/step -
accuracy: 0.8850 - loss: 0.2725 - val_accuracy: 0.7867 - val_loss: 0.4005
Epoch 27/30
2/2             1s 741ms/step -
accuracy: 0.9083 - loss: 0.2350 - val_accuracy: 0.9133 - val_loss: 0.2591
Epoch 28/30
2/2             1s 734ms/step -
accuracy: 0.9417 - loss: 0.1931 - val_accuracy: 0.8933 - val_loss: 0.2429
Epoch 29/30
2/2             1s 723ms/step -
accuracy: 0.9217 - loss: 0.2230 - val_accuracy: 0.9200 - val_loss: 0.2213
Epoch 30/30
2/2             1s 723ms/step -
accuracy: 0.9450 - loss: 0.1806 - val_accuracy: 0.8933 - val_loss: 0.2329
5/5             0s 23ms/step -
accuracy: 0.8933 - loss: 0.2329
Test Accuracy: 0.8933333158493042
```

[72]:
```python
import matplotlib.pyplot as plt

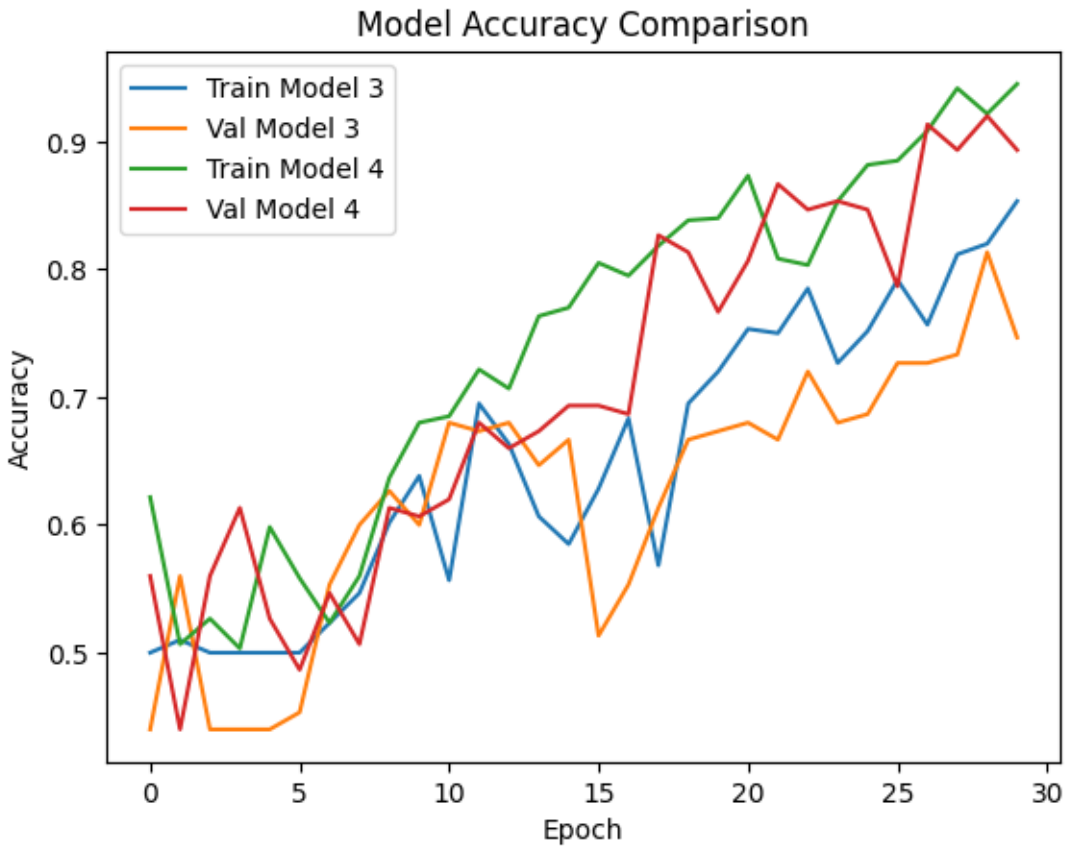
# Accuracy
plt.plot(history3.history['accuracy'], label='Train Model 3')
plt.plot(history3.history['val_accuracy'], label='Val Model 3')
plt.plot(history4.history['accuracy'], label='Train Model 4')
plt.plot(history4.history['val_accuracy'], label='Val Model 4')
```

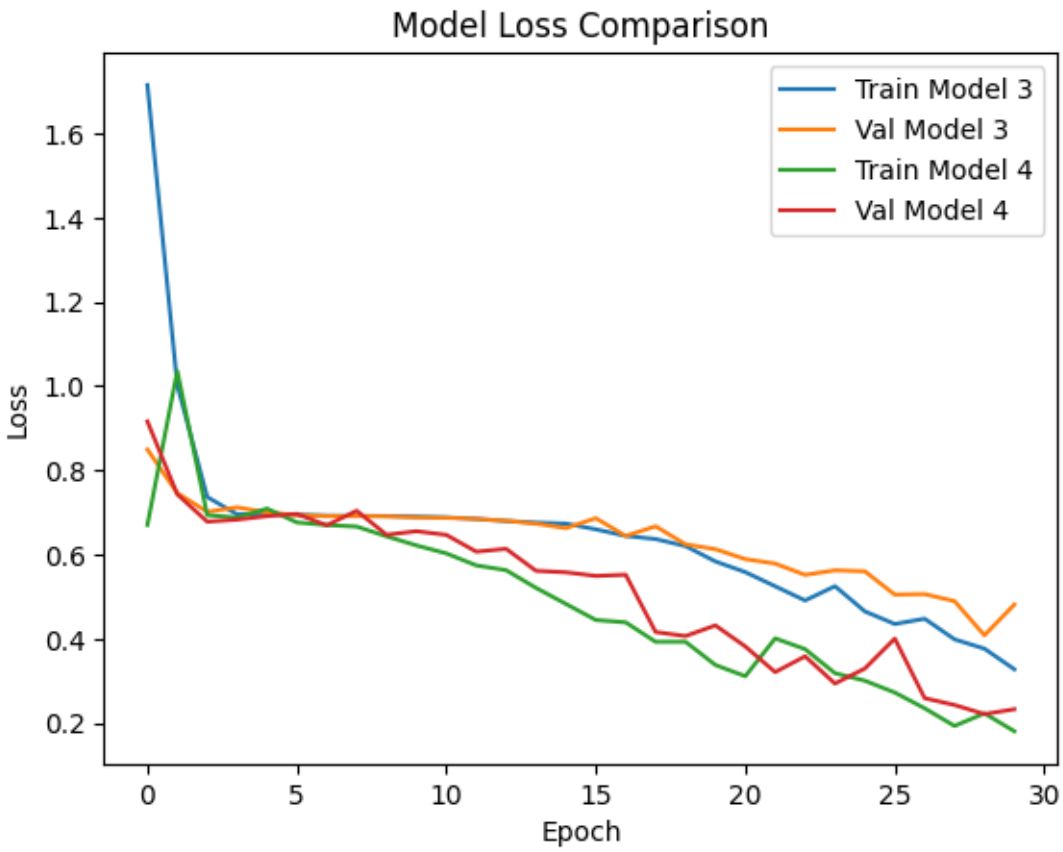
```
plt.title("Model Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()


# Loss

plt.plot(history3.history['loss'], label='Train Model 3')
plt.plot(history3.history['val_loss'], label='Val Model 3')
plt.plot(history4.history['loss'], label='Train Model 4')
plt.plot(history4.history['val_loss'], label='Val Model 4')

plt.title("Model Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```





The two models show similar behavior with good performance on the test set. Model 3 achieves an accuracy of 0.94, while Model 4 reaches 0.84. The loss for both models decreases during training, which indicates stable learning and convergence.

Model 3 produces the best results even with fewer dense layers. This suggests that a simpler classifier can generalize better for this task. Model 4 contains more dense layers and different activations, which increases model complexity and leads to slightly lower accuracy on the test set.